Strumenti per programmare in Go

I comandi Go

L’installazione di Go include vari strumenti per compilare, eseguire e formattare il codice, oltre ad altri strumenti di utilità.

- `go fmt`: formatta il codice di un singolo file o di un intero package
- `go run <programma.go>`: compila ed esegue un programma Go
- `go build`: compila un package creando un file eseguibile
- `go doc`: restituisce la documentazione relativa ad un package Go o ad una funzione
- `go help <comando>`: restituisce la documentazione relativa ad un comando Go

Go fmt

```
$ ls
main.go

$ go fmt main.go
```

Go fmt (2)

Codice prima della formattazione:

```
package main
import "fmt"
func main() {
fmt.Println("Hello world!")
     }
```

Codice dopo la formattazione

```
package main

import "fmt"

func main() {
    fmt.Println("Hello world!")
}
```

Go doc

```
$ go doc fmt.Println
package fmt // import "fmt"

func Println(a ...interface{}) (n int, err error)
  Println formats using the default formats for its operands a
  standard output. Spaces are always added between operands an
  appended. It returns the number of bytes written and any wri
  encountered.
```

Go run

```
$ ls
main.go

$ go run main.go
Hello world!
```

```go
package main

import "fmt"

func main() {
    fmt.Prinln("Hello world!")
}
```

Il comando `go mod init`

Perché si usa?

- Indica a Go la radice del progetto.
- Tiene traccia delle librerie esterne (gestione delle dipendenze) e della versione di Go usata.
- Permette di compilare ed eseguire il codice in qualsiasi cartella del computer (senza vincoli di `$GOPATH`).

Come funziona?

Crea un file chiamato `go.mod` nella directory corrente.

```
$ go mod init <nome_cartella>
go: creating new go.mod: module <nome_cartella>
```

Go build

```
$ ls
main.go
```

1. Create Go module

```
$ go mod init helloWorld
```

2. Build the module

```
$ go build -o Hello
```

```
$ ls
Hello main.go
```

```
$ ./Hello
Hello world!
```

Go help

```
$ go help build
usage: go build [-o output] [-i] [build flags] [packages]

Build compiles the packages named by the import paths,
along with their dependencies, but it does not install the res

If the arguments to build are a list of .go files from a singl
build treats them as a list of source files specifying a singl

When compiling packages, build ignores files that end in '_tes

...
```

Esercizi

Esercizio 1

Usare gli strumenti `go run` e `go build` per eseguire il codice `helloworld.go`.

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello World!")
}
```

Esercizio 2

Utilizzate lo strumento `go fmt` per formattare in modo automatico il codice `formattazione.go`.

```
package main
import "fmt"
func main() {
var x int
    x = 10
            var y int
y = 10
    fmt.Println(x+y     )
     }
```

Esercizio 3

Lo strumento go `fmt` non è in grado di formattare il codice quando sono presenti degli errori. È quindi importante formattare il proprio codice durante la scrittura.

Correggere gli errori del seguente codice, formattarlo ed eseguirlo.

```
package main
import fmt

func main {
var x int
x = 10
var y int
y = 15
sum = x + y
fmt.Println("La somma è ", sum
}
}
```

Esercizio 4

Utilizzare i comandi della shell Bash e gli strumenti Go per svolgere le seguenti operazioni:

1. Creare una struttura di directory `lab1/rettangolo` e posizionarsi al suo interno.
2. Inizializzare un modulo Go denominato `rettangolo`.
3. Creare un file `main.go` che definisca due variabili per la **base** (15) e l'**altezza** (7) di un rettangolo.
4. Il programma deve calcolare e stampare a schermo:
   - Il perimetro del rettangolo
   - L'area del rettangolo
5. Compilare il programma generando un file eseguibile chiamato `area_calcolo` (usando l'opzione `-o` di `go build`).
6. Eseguire l'eseguibile appena creato da terminale.

Esercizio 5

Il seguente codice Go dovrebbe calcolare quanti secondi ci sono in **3 giorni, 4 ore e 15 minuti**, ma contiene diversi errori di sintassi, problemi di tipo e una formattazione errata.

1. Creare il file `tempo.go` e incollarvi il codice fornito.
2. Identificare e correggere gli errori di sintassi affinché il programma possa essere compilato.
3. Formattare il codice utilizzando lo strumento `go fmt`.
4. Eseguire il programma corretto tramite `go run`.

```
package main
import fmt

func main {
var giorni int = 3
var ore = 4
var minuti int = 15

   totaleOre := giorni * 24 + ore
  totaleMinuti := totaleOre * 60 + minuti
     totaleSecondi := totaleMinuti * "60"

fmt.println("Totale secondi:" totaleSecondi
}
```

Esercizio 6

1. Creare un programma `ricevuta.go` che stampi a schermo una semplice ricevuta di calcolo usando più istruzioni di stampa standard (`fmt.Println`):

```
==================================
LABORATORIO DI PROGRAMMAZIONE 1
==================================
Prezzo unitario: 25 EUR
Quantita: 4
Sconto applicato: 10 EUR
Totale finale: 90 EUR
==================================
```

Nota: Il totale finale ($25 \times 4 - 10$) deve essere calcolato direttamente come espressione aritmetica all'interno del programma Go e stampato insieme al testo.

2. Eseguire il programma reindirizzando l'output generato su un file di testo chiamato `ricevuta.txt`.

3. Verificare il contenuto del file `ricevuta.txt` direttamente dal terminale stampandone il contenuto.

Esercizio 7

Un agricoltore raccoglie **143 mele** e vuole distribuire le mele in cassette da **12 mele ciascuna**.

1. Creare un programma `mele.go` che definisca due variabili:
   - `totaleMele` (143)
   - `capienzaCassetta` (12)
2. Utilizzando unicamente le operazioni aritmetiche di base (`+`, `-`, `*`, `/`), calcolare e stampare:
   - Il numero di cassette piene che si possono riempire (sfruttando la divisione intera `/`).
   - Il numero di mele avanzate che non completano una cassetta (calcolabile come differenza tra il totale e le mele già distribuite nelle cassette).
3. Formattare il codice con `go fmt` prima di eseguirlo con `go run`.

Esercizio 8

1. Creare la directory `spazio` e inizializzare un modulo Go chiamato `spazio`.
2. Scrivere un programma `countdown.go` che calcoli i secondi totali mancanti al lancio di un razzo, sapendo che mancano **2 ore, 45 minuti e 30 secondi**. Il programma deve stampare a schermo la frase: `T-minus <totale_secondi> secondi al lancio!`.
3. Creare una sottocartella chiamata `bin` all'interno della cartella `spazio`.
4. Compilare il programma in modo che il file eseguibile denominato `rocket` venga generato direttamente dentro la sottocartella `bin` (`bin/rocket`).
5. Verificare la presenza del file binario con il comando `ls bin/` ed eseguirlo indicando il percorso relativo.